Reviewer Pack — Minimal Rank of Tropicalized Cohomology Groups over Branchin…

Entry 2f471259c7d3 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-25 21:29:31 UTC

Minimal Rank of Tropicalized Cohomology Groups over Branching Programs

Entry ID: `2f471259c7d3` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-25 21:29:31 UTC

1. Conjecture

Field A (mathematical branch): Tropical Geometry (Cohomology) **Field B** (complexity object): Complexity Theory: Branching Program Complexity

Statement:

> {'sentence_1': 'For every branching program P with read-twice access, the minimal rank of its tropicalized cohomology groups is O(log n), where n is the size of P.', 'sentence_2': 'However, for an IP_2 trivial branching program, this rank is at least Ω(n^(1/3)).', 'sentence_3': 'This relationship holds even after accounting for polynomial factors.'}

Rationale (proposer's reasoning):

> {'sentence_1': 'Tropical cohomology provides a natural way to study the algebraic structure of branching programs.', 'sentence_2': 'The minimal rank of tropicalized cohomology groups could potentially reveal hidden complexities within branching program computation that are not captured by their size alone.', 'sentence_3': 'By proposing a quantitative relation between these two areas, we aim to uncover new structural insights into the complexity of branching programs.'}

Taxonomy category: `BP_READTWICE` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `c7d15f0292d21ff7`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The conjecture is supported if, for all generated read-twice branching programs P with size n ≤ 40, the minimal rank of tropicalized cohomology groups falls within O(log n) with at least 80% confidence and no seed produces a rank exceeding 2 * log(n)^1.5. It is falsified if any seed produces a rank exceeding 2 * log(n)^1.5 or the support fraction is below 0.8.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 7 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - `tropical geometry AND cohomology AND branching programs` - `branching program complexity AND tropicalized cohomology groups` - `minimal rank IN Tropical Geometry AND ON Branching Programs`

Top relevant hits considered: - [http://arxiv.org/abs/1207.2443v2] Tropical Teichmuller and Siegel spaces - [http://arxiv.org/abs/1204.3875v2] Tropicalizing vs Compactifying the Torelli morphism - [http://arxiv.org/abs/1204.6154v2] Local Tropicalization - [http://arxiv.org/abs/2106.11479v4] Differential forms and cohomology in tropical and complex geometry - [http://arxiv.org/abs/math/0209219v2] On the vanishing of the measurable Schur cohomology groups of Weil groups - [http://arxiv.org/abs/2006.03470v On subset sum problem in branch groups - [http://arxiv.org/abs/1610.00298v4] Khovanskii bases, higher rank valuations and tropical geometry

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def gaussian_elimination(A):
        m, n = len(A), len(A[0])
        for i in range(m):
            max_row = i
            for j in range(i+1, m):
                if abs(A[j][i]) > abs(A[max_row][i]):
                    max_row = j
            A[i], A[max_row] = A[max_row], A[i]
            factor = A[i][i]
            for j in range(n):
                A[i][j] /= factor
            for j in range(m):
                if i != j:
                    factor = A[j][i]
                    for k in range(n):
                        A[j][k] -= factor * A[i][k]
```

```
        return A

    def matrix_multiplication(A, B):
        m, n, p = len(A), len(B), len(B[0])
        C = [[0] * p for _ in range(m)]
        for i in range(m):
            for j in range(p):
                for k in range(n):
                    C[i][j] += A[i][k] * B[k][j]
        return C

    def log2(x):
        if x <= 0:
            return float('-inf')
        return math.log2(x)

    n = random.randint(5, 40)
    P = [[random.choice([0, 1]) for _ in range(n)] for _ in range(n)]

    # Compute tropicalized cohomology groups
    A = []
    for i in range(n):
        row = [max(P[j][i] for j in range(n)) for i in range(n)]
        A.append(row)

    A_tilde = gaussian_elimination(A)
    rank = sum(1 for row in A_tilde if any(x != 0 for x in row))

    metric_value = rank
    instances_tested = 1
    conjecture_holds = rank <= 2 * log2(n) ** 1.5
    counterexample = "" if conjecture_holds else f"Rank {rank} exceeds 2 * log({n})^1.5"

    return {
        "metric_name": "Minimal Rank of Tropicalized Cohomology Groups",
        "metric_value": metric_value,
        "instances_tested": instances_tested,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] if len(sys.argv) > 1 else [2**i - 1 for i in range(5, 8

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    total_metric_value = sum(result["metric_value"] for result in results)
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if all(result["conjecture_holds"] for result in results):
        print(f"RESULT: SUPPORTED mean={total_metric_value/len(results):.2f} std=0.00 support_frac
```

```
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={total_metric_value/len(results):.2f} std=0.00 support_frac
    else:
        first_failing_seed = next(seed for seed, result in enumerate(results) if not result["conje
        print(f"RESULT: FALSIFIED counterexample=\"Rank exceeds 2 * log(n)^1.5\" first_failing_see
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_60d7cb87.py", line 84, in <module>
    trial_result = run_trial(seed)
                   ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_60d7cb87.py", line 62, in run_trial
    A_tilde = gaussian_elimination(A)
              ^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_60d7cb87.py", line 31, in gaussian_elim
    A[i][j] /= factor
ZeroDivisionError: float division by zero
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> The test crashed before producing data, which means the pre-registered support condition could not be unambiguously met. | next: Re-run the test without crashing to verify the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	12652
2	propose	ollama_remote	glm4:latest	0	0	14667
3	preregistration	ollama_remote	glm4:latest	0	0	5963
4	novelty	ollama_remote	glm4:latest	0	0	4481
5	novelty	ollama_remote	glm4:latest	0	0	5772
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14781
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10118
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7387
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9769

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
10	judge	ollama_remote	glm4:latest	0	0	11327

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 96917 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in `research/audit/2f471259c7d3.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/2f471259c7d3.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/2f471259c7d3.tar.gz` (if generated)